

Sorting

ผศ.จรรยา สายწყຍ

344-111 ชุคควิชาการโปรแกรมและชั้นตอนวิธี

Content

- ▶ **Bubble Sort**
- ▶ **Selection Sort**
- ▶ **Insertion Sort**
- ▶ **Shell Sort**
- ▶ **Merge Sort**
- ▶ **Quick Sort**



ผลการเรียนรู้ที่คาดหวัง

- ▶ เข้าใจการทำงานขั้นตอนวิธีการจัดเรียงแต่ละวิธี
- ▶ เขียนขั้นตอนวิธีของการจัดเรียงแต่ละวิธีได้
- ▶ วิเคราะห์เวลาในการทำงานของแต่ละขั้นตอนวิธีได้



Bubble Sort

- ▶ **Brute Force Algorithm**
- ▶ **Idea:** to compare adjacent elements of the list and exchange them if they are out of order



Bubble Sort (cont.)

► Example

i	0	1	2	3	4	5	swapped
Data	30	54	27	70	60	22	1
Round 1	30	27	54	60	22	70	1
Round 2	27	30	54	22	60	70	1
Round 3	27	30	22	54	60	70	1
Round 4	27	22	30	54	60	70	1
Round 5	22	27	30	54	60	70	1
Round 6	22	27	30	54	60	70	0

Bubble Sort (cont.)

► Algorithm:

```
BubbleSort(data[1..n], n){
    swapped = 1;
    while(swapped==1){
        swapped = 0;
        for (i=1; i<n ;i++ ){
            if(data[i] > data[i+1]){
                tmp = data[i];
                data[i] = data[i+1];
                data[i+1]=tmp;
                swapped = 1;
            }
        }
    }
}
```

► Time Complexity: ??



Selection sort

- ▶ **Brute Force Algorithm**
- ▶ **Idea:**



Selection sort (cont.)

► Example

i	0	1	2	3	4	5
Data	30	54	27	70	60	22
Round 1	22	54	27	70	60	30
Round 2	22	27	54	70	60	30
Round 3	22	27	30	70	60	54
Round 4	22	27	30	54	60	70
Round 5	22	27	30	54	60	70
Round 6	22	27	30	54	60	70

Selection sort (cont.)

► Algorithm:

```
SelectionSort(data[1..n], n){  
    for(i=1; i<n; i++){  
        tmp = data[i]; k=i;  
        for(j=i+1; j<n; j++){  
            if(data[j]<tmp){  
                tmp = data[j];  
                k=j;  
            }  
        }  
        tmp = data[i];  
        data[i]=data[k];  
        data[k]=tmp;  
    }  
}
```

► Time Complexity:



Insertion sort

► **Problem:** to sort an array $a[1..n]$

► **Idea:**

- assume that the smaller problem of the array $a[1..n-1]$ has already sorted.
- e.g., if $a[1..i]$ has already sorted, then an element $a[i+1]$ is inserted in its proper position among $a[1..i]$

$$\underbrace{a[1] \leq \cdots \leq a[j] \leq \cdots \leq a[i]}_{\text{Sorted array}} \mid a[i+1] \dots a[n]$$



$$a[i+1] < a[j]$$



Insertion sort

i	1	2	3	4	5	6	temp
Data	30	54	27	70	60	22	
Round 1	30	54	27	70	60	22	54
Round 2	27	30	54	70	60	22	27
Round 3	27	30	54	70	60	22	70
Round 4	27	30	54	60	70	22	60
Round 5	22	27	30	54	60	70	22



Insertion sort

► Algorithm:

```
InsertionSort(data[1..n], n){  
    for(i=2; i<=n; i++){  
        tmp = data[i];  
        j=i;  
        while(j > 1 && data[j-1]>tmp){  
            data[j] = data[j-1];  
            j=j-1;  
        }  
        data[j]=tmp;  
    }  
}
```

► Time Complexity?



Insertion sort

Time Complexity:

► Worst-case:

$$T(n) = \sum_{i=2}^n \sum_{j=2}^i 1 = \sum_{i=2}^n (i - 1) = \Theta(n^2)$$

► Best-case:

$$T(n) = \sum_{i=2}^n 1 = \Theta(n)$$

► Average-case:

$$T(n) = \sum_{i=2}^n \left(\frac{1}{i} \sum_{j=1}^i j \right) = \sum_{i=2}^n \left(\frac{i+1}{2} \right) = \Theta(n^2)$$



Shell sort

► **Idea:**

1. Break the list into disjoint subsets
2. Sort them using insertion sort
3. Combine the sorted subsets



Shell sort (cont.)

► Example:

n = 7	0	1	2	3	4	5	6
DATA ; รอบที่ 1 $D = \text{round}(7/2) = 4$	10	35	73	25	23	27	97
ผลจากการจัดเรียง รอบที่ 1	10	27	73	25	23	35	97

n = 7	0	1	2	3	4	5	6
DATA ; รอบที่ 2 $D = \text{round}(4/2) = 2$	10	27	73	25	23	35	97
ผลจากการจัดเรียง รอบที่ 2	10	25	23	27	73	35	97

n = 7	0	1	2	3	4	5	6
DATA ; รอบที่ 2 $D = \text{round}(2/2) = 1$	10	25	23	27	73	35	97
ผลจากการจัดเรียง รอบที่ 3	10	23	25	27	35	73	97

Shell sort (cont.)

► Algorithm:

```
for (D =  $\lceil \frac{n}{2} \rceil$ ; D > 0; D =  $\lceil \frac{D}{2} \rceil$ )  
{  
  for (i = 1 + D to n)  
  {  
    j = i; tmp = data[j]  
    while (j > 0 && data[j-D] > tmp)  
    {  
      data[j] = data[j-D]  
      j = j - D  
    }  
    data[j] = tmp  
  }  
}
```

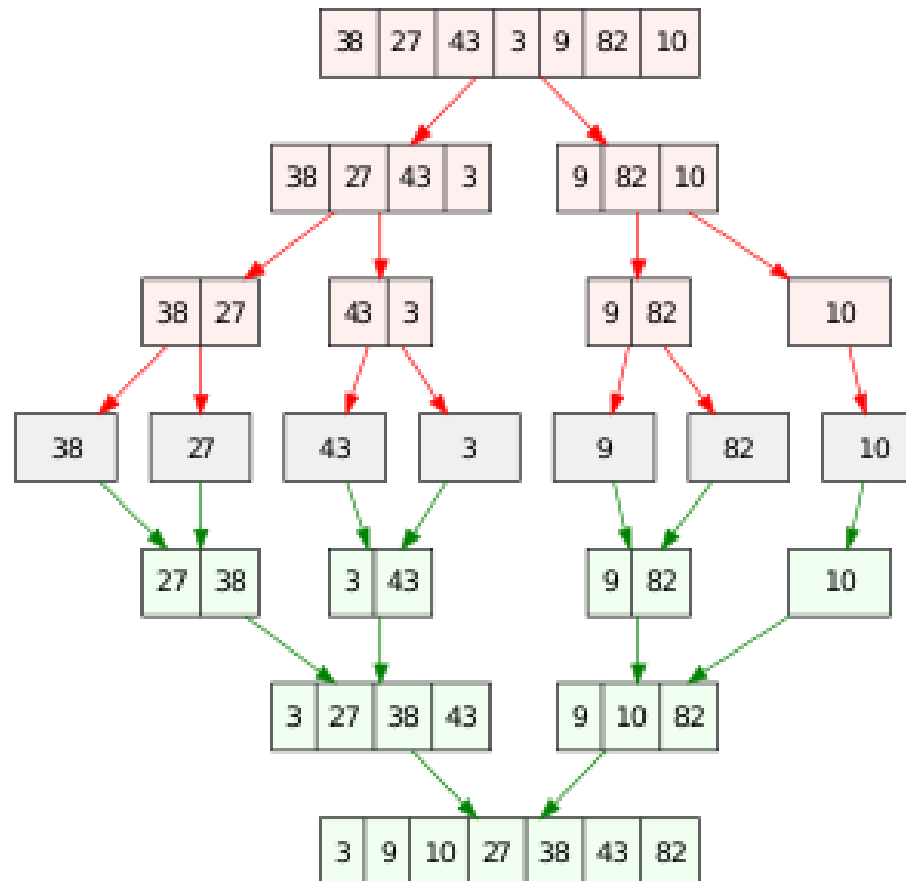
► Time complexity:

Merge Sort

- ▶ **Concept:**
- ▶ 1) **dividing the data into two subsets** of the same size
- ▶ 2) sorting each of them recursively
- ▶ 3) **merging the two smaller sorted** into the a single sorted.



Merge Sort



Algorithm: Merge sort

MergeSort (A, p, r)

{

 if ($p < r$) $q = (p+r)/2$

MergeSort (A, p, q)

MergeSort (A, q+1, r)

 Merge (A, p, q, r) // merges A[p..q] with A[q+1..r]

}

Time Complexity:

- $t(n) = 2t(n/2) + \Theta(n) \in \Theta(n \log n)$
- $t_{\text{worst}} = t_{\text{avg}} = t_{\text{best}}$



Algorithm: Merge

```
MERGE( $A, p, q, r$ )
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Time Complexity is $O(n)$

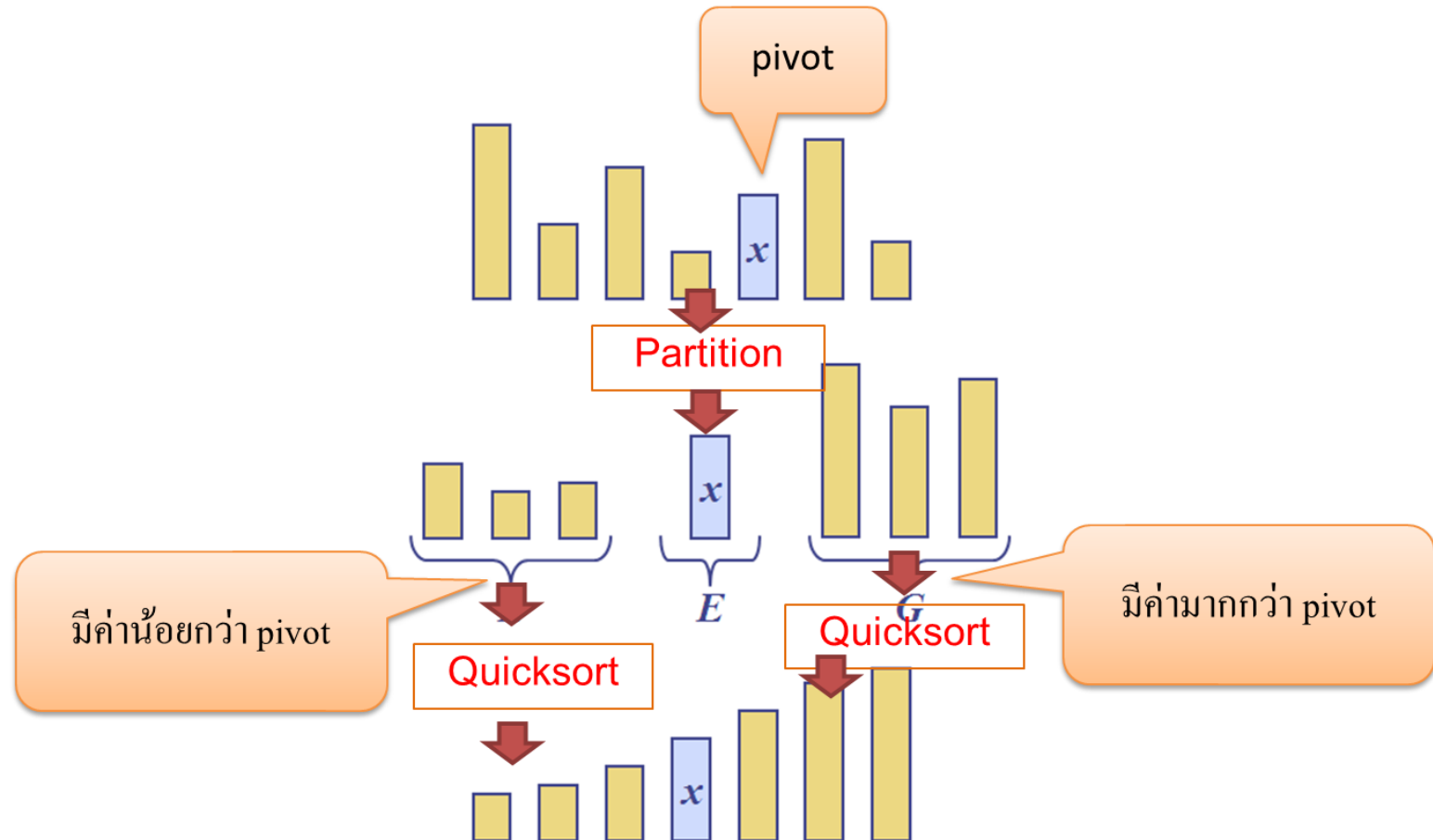


Quick sort

- ▶ **Concept:**
- ▶ 1) selecting a data, called **pivot**, from data points
- ▶ 2) **partitioning the data into two sets**; the left and the right of the pivot,
 - ▶ Left: the data that are smaller than the pivot
 - ▶ Right: the data that are greater than the pivot
- ▶ 3) **recursive** step 1) and step 2) on the two subsets independently



Quick sort



Algorithm: Quicksort

QuickSort(A, p, r)

```
{  
  if (p < r){  
    j = Partition(A, p, r)  
    QuickSort(A, p, j)  
    QuickSort(A, j+1, r)  
  }  
}
```

Partition(A, p, r)

```
{  
  c = A[p]  
  i = p-1; j = r+1  
  while (i < j){  
    while(A[--j] > c) { }  
    while(A[++i] < c) {}  
    if (i < j) swap(A[i], A[j])  
  }  
  return j  
}
```

Time Complexity in average
cased is $O(n \log n)$



ศึกษาเพิ่มเติม

- ▶ การจัดเรียงข้อมูลแบบ **Radix Sort**

- ▶ <https://www.youtube.com/watch?v=GUHGMtNo6RQ>

- ▶ <https://www.youtube.com/watch?v=IA9N4I5CCFU>

- ▶ การจัดเรียงข้อมูลแบบ **Merge Sort**

- ▶ <https://www.youtube.com/watch?v=GCaeIWNvnZM>

- ▶ การจัดเรียงข้อมูลแบบ **Quick Sort**

- ▶ <https://www.youtube.com/watch?v=ASrmngOOIr4>

- ▶ https://www.youtube.com/watch?v=y_G9BkAm6B8

- ▶ <http://sci.feu.ac.th/faa/dsa/bookPDFs/chap5-Sorting.pdf>

- ▶ http://www.elfhs.ssru.ac.th/wipada_ch/pluginfile.php/893/course/summary/Chapter%2013-Sorting.pdf

